UniSync

Progress Documentation

System Development and Repository Deployment

COMP 018: Database Administration · Group 10

Item	Details
Group	10 – UniSync: A University Event Scheduling and Registration System
Members	Cofreros, Joshua Rei – Lead Developer: Backend, Frontend and Database implementation Aranguste, Jerome – Documentation Lead: README, user roles and setup documentation Patrona, John Timothy – Database Documentation: ERD, schema and database documentation, progress report
Repository	github.com/CoffyJosh/unisync (branch to test: `main`)
Class tracker	docs.google.com/spreadsheets/d/1BClCa9rEs9o-D9Rp3tUZVhECipJ234zX
Tech stack	PHP 8.1+ (PDO), PostgreSQL 18.6, HTML, Tailwind CSS, JavaScript, jQuery (AJAX), Leaflet
Date submitted	7 November 2026

1. System Overview

UniSync is a web application where university students discover organizations, request to join them, browse events, and register for them. Organization Admins review join requests and manage the organization, its members, and its events. Managers help run the organization's events. Its purpose is to centralize student organization membership and event registration in one system backed by a normalized PostgreSQL database, as proposed in Activity 2.

Main functions

- Account registration and login
- Create organizations (the creator becomes Admin)
- Request to join organizations (requests stay pending until an Admin reviews them)
- Admins accept or reject join requests
- Cancel your own pending join request
- Leave organizations, with automatic Admin succession
- Role management and member removal (Admin and Manager)
- Edit and delete organizations (Admin)
- Create, edit and delete events with multiple schedules and map-pinned venues
- Event status derived from schedules (Pending, Confirmed, Completed, Cancelled)
- Live countdown to the next confirmed schedule
- Event registration and cancellation
- Attendee list with removal for Admins and Managers

2. User Roles

Application roles describe what a person can do after logging in; they are separate from PostgreSQL roles. End users do not get their own database login: the application connects through a single runtime account. Organization roles (Admin, Manager, Member) are stored per organization in MEMBERSHIP, so one account can hold different roles in different organizations. UniSync has no system-wide administrator inside the application: all authority is scoped to an organization, and Organization Admin only applies to that one organization. System Administrator, Application runtime and Viewer / auditor are database roles outside the application.

Application role	Responsibilities	Functions and data access	Intended DB role
Guest (not logged in)	Browses public pages	Read organizations and events	Runtime account
Student / Member	Requests to join organizations, registers for events	Submit and cancel own join requests; create and update own membership and registrations; read organizations and events	`unisync_runtime_grp`
Organization Manager	Runs events for the organization	Member functions plus create, update and delete events and schedules; view and remove attendees; remove regular Members; cannot review join requests	`unisync_runtime_grp`
Organization Admin	Administers the organization	Manager functions plus accept or reject join requests, edit and delete the organization, change member roles, remove any member	`unisync_runtime_grp`
System Administrator	Maintains the database outside the application (running init.sql and seed.sql, backups, role and privilege setup)	Not a user of the web app and has no application login; full administrative access to UniSync database objects, not a superuser	`unisync_admin_grp`
Application runtime (system account)	Account PHP uses to connect	SELECT, INSERT and UPDATE on application tables; no DDL	`unisync_runtime_grp`
Viewer / auditor	Reviews records and reports	Read-only on approved tables and views	`unisync_readonly_grp`

Database login accounts and privileges per group role are documented in `docs/database-roles.md`.

3. Development Progress Log

Dates and commits come from `git log`.

Date	Milestone	Who	Key commits	What changed
2026-10-30	Repository setup	Joshua	ae096d0, 24341ab, 8b54333, 8d30922, 241ab69, e8e3a5d	Initial upload of 17 files; README drafted and revised.
2026-10-31	Core pages	Joshua	750737f, da21abb, a302a4e	Homepage and browse page; core features across 16 files; bug fixes.
2026-11-04	Dashboard and organization page	Joshua	d84086d	23 files, +4,775 lines.

Date	Milestone	Who	Key commits	What changed
2026-11-05	Accounts, events, schema	Joshua	2467dd5	42 files, +6,264 lines. Reworked `init.sql` and `seed.sql`; student-to-account migration script; separate account and provider login and registration; create-event and create-organization pages, handlers and JS.
2026-11-05	Event editing	Joshua	fd3e3c9	Added edit_event (page, JS) and `update_event.php`; updated browse results and event pages.
2026-11-05	Organization management	Joshua	74f1e13, 828b509	Added `delete_organization.php`; changed kick_member, update_member_role, join and leave handlers; reworked the organization page and event.js.
2026-11-05	Cleanup	Joshua	c70f57c, 661a9c7, 3a0721a	Small removals and refactors.
2026-11-06	Repository cleanup	Joshua	38d4797, 5f9e48a, 47337f8	Removed 12 test upload images; moved javascript/ and assets/ into public/assets/ and updated page paths; added leaflet.js; removed TODO.md.
2026-11-06	Database and role documentation	Joshua	d6cb7e3, a2929d8	Added docs/database.md, user-roles.md, database-roles.md, troubleshooting.md; removed the migration script from db/.
2026-11-06	README, samples, small fixes	Joshua	967bdc2, 7f30617	Sample accounts, the "undefined prompt" fix, README revisions.

4. Database Implementation

Database: `grp10_unisyncdb` · **DBMS:** PostgreSQL 18.6 · **Schema:** `public` · **Tables:** 8. `db/init.sql` runs in one transaction and drops and recreates all tables, so it must be run before `db/seed.sql`.

4.1 Entity Relationship Diagram

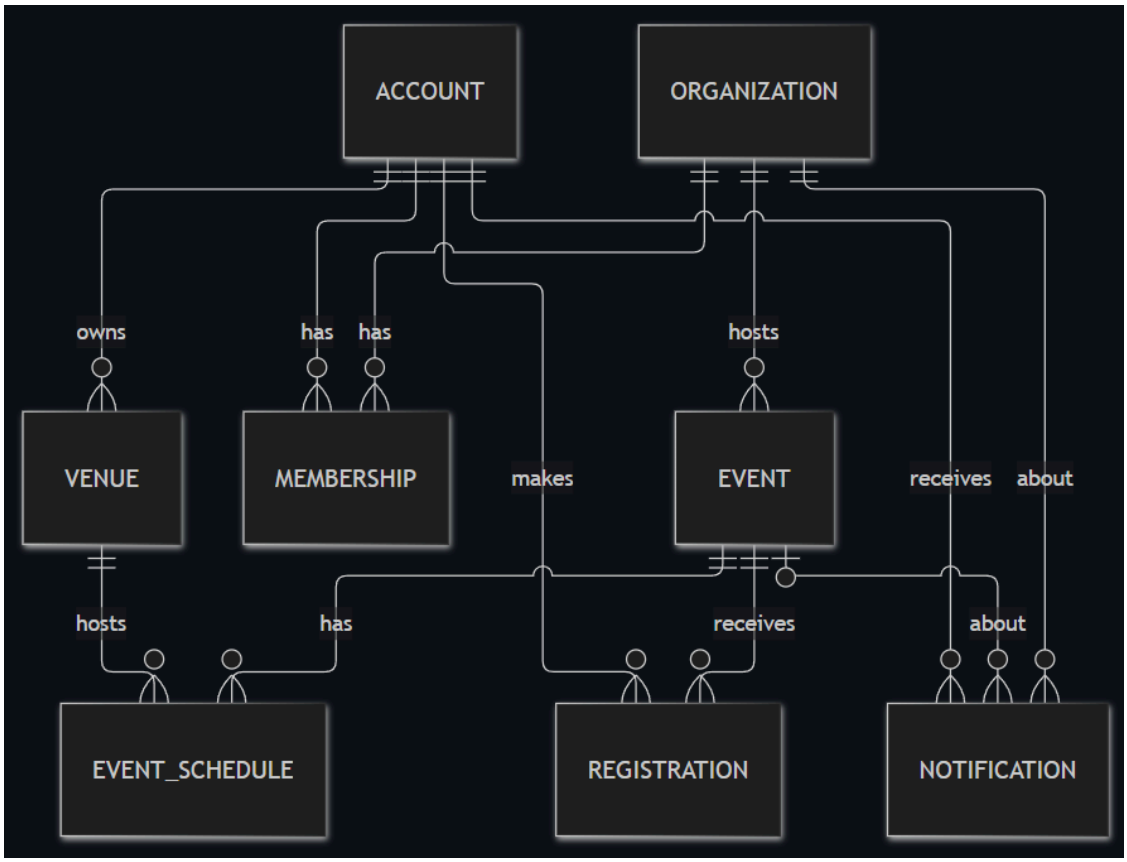


Figure 1. UniSync entity relationship diagram (8 tables, 11 relationships)

4.2 Schema Summary

Table	Purpose	Keys and key constraints
ACCOUNT	User accounts; a venue is tied to the account that created its schedule	PK account_id; UNIQUE email; status in (active, disabled, deleted)
ORGANIZATION	University organizations	PK organization_id; status in (active, inactive, deleted)
MEMBERSHIP	Links accounts to organizations; stores the organization-level role and join-request status	PK membership_id; FKs to ACCOUNT and ORGANIZATION (CASCADE); UNIQUE (account_id, organization_id); role in (Admin, Manager, Member); status in (pending, active, rejected, inactive, deleted); a pending or rejected request must have role Member
EVENT	Events created by organizations	PK event_id; FK organization_id (CASCADE); status in (pending, confirmed, cancelled, completed, deleted)
VENUE	Event locations with coordinates	PK venue_id; FK account_id (RESTRICT); UNIQUE (account_id, venue_name); capacity > 0; latitude and longitude in range; both set or both NULL
EVENT_SCHEDULE	Date, time and venue per event; also the venue application record	PK schedule_id; FK event_id (CASCADE), venue_id (RESTRICT); end_time > start_time; status in (pending, confirmed, rejected, cancelled, completed)
REGISTRATION	Account-to-event registrations	PK registration_id; FKs CASCADE; UNIQUE (account_id, event_id); status in (active, cancelled, deleted)
NOTIFICATION	In the schema but not used by the application	PK notification_id; FKs to ACCOUNT and ORGANIZATION (CASCADE) and EVENT (SET NULL)

Business rules. One membership per account per organization; joining is a request that creates a pending membership, which only an Admin can accept (it becomes active) or reject (it becomes rejected); the requester can cancel a pending request and a rejected user can request again; a pending or rejected request never carries an elevated role; one registration per account per event; a schedule's end time must be after its start time; venues cannot be hard-deleted while schedules or their owner account depend on them. The application uses soft deletes through a status column (deleted, inactive, cancelled), and deleting an organization also marks its events as deleted.

Indexes. Ten indexes: separate single-column indexes on MEMBERSHIP (organization_id) and (account_id), a partial index on MEMBERSHIP (organization_id) for pending join requests, EVENT (organization_id), VENUE (account_id), REGISTRATION (event_id) and (account_id); EVENT_SCHEDULE (event_date, venue_id) and (status); and NOTIFICATION (account_id, created_at DESC). No views, triggers or functions are defined in `init.sql`.

4.3 Scripts and Seed Data

- `db/init.sql` creates the schema, constraints and indexes.
- `db/seed.sql` loads 100+ accounts (hashed sample passwords), 30 organizations, 50 events, and venues with real addresses and map coordinates. NOTIFICATION is intentionally empty.

4.4 Changes from the Approved Design

Change	Reason
STUDENT table became ACCOUNT	One table serves all users, since roles are now stored per organization
VENUE gained account_id and UNIQUE (account_id, venue_name); venue name is no longer globally unique	Each venue is created with an event schedule and tied to the account that created it, so different accounts may use the same venue name
Status columns added to MEMBERSHIP, VENUE, EVENT_SCHEDULE and REGISTRATION; soft deletes	Keeps history and avoids breaking related records; a removal can be undone
New columns: password_hash, avatar_url, banner_url, event_image	Support login, profile and organization images, and event images
Organization role stored per membership	A user can hold a different role in each organization
Join requests stored in MEMBERSHIP with a status (pending, active, rejected, inactive, deleted)	Joining is a request that an Admin must approve instead of an immediate join, so no separate request table is needed
NOTIFICATION table added but not implemented	Planned for alerts, but not finished before the deadline; it is in the schema but unused by the app
Planned 7 tables and 7 relationships became 8 tables and 11 relationships	Result of the changes above

5. Application Evidence

Create, retrieve, update and delete operations were tested on the running system. All tests passed.

Operation	Test performed	Expected result	Result	Figure
Create	Register a new account	Account row inserted	Pass	Fig. 2

Operation	Test performed	Expected result	Result	Figure
Create	Create an organization	Organization created; creator is Admin in MEMBERSHIP	Pass	Fig. 3
Create	Create an event with schedule and map-pinned venue	EVENT, VENUE and EVENT_SCHEDULE rows created	Pass	Fig. 4
Create	Register for an event	REGISTRATION row; duplicate blocked	Pass	Fig. 5
Retrieve	Browse events and organizations; search	Correct results; no other organization's events shown	Pass	Fig. 6
Retrieve	Open event page with countdown and attendee list	Correct details shown	Pass	Fig. 7
Update	Edit organization and event; change a member's role	Changes saved	Pass	Fig. 8
Delete	Cancel a registration; remove a member; delete an event or organization	Status set to cancelled or deleted (soft delete)	Pass	Fig. 9

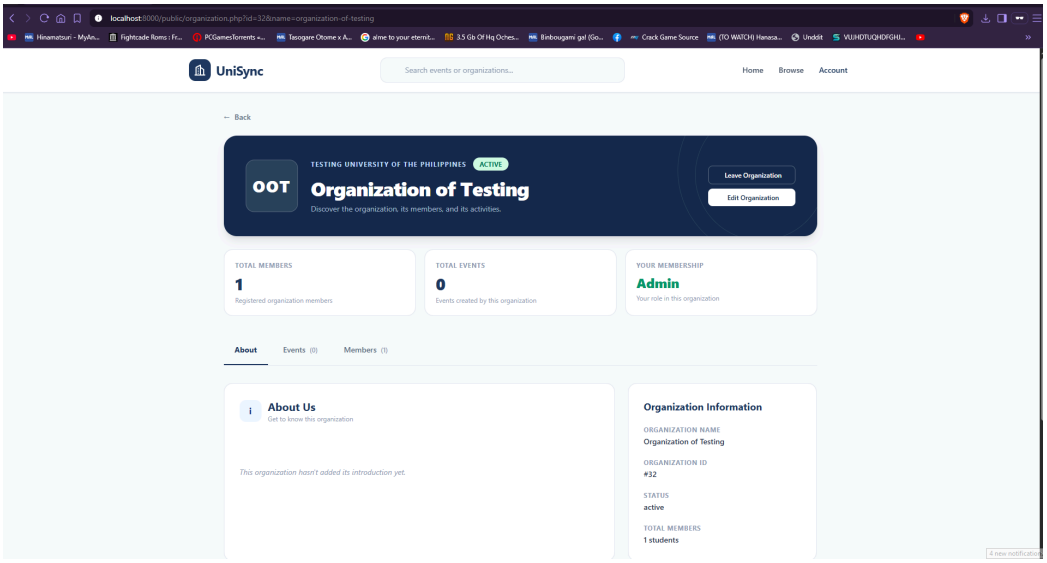


Figure 2. Account created successfully

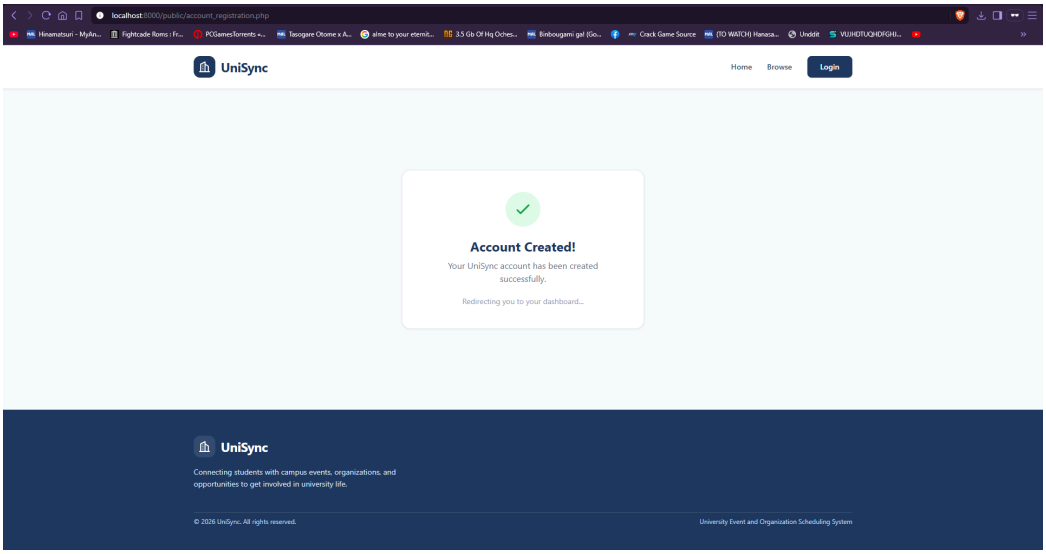


Figure 3. Organization created; creator is Admin

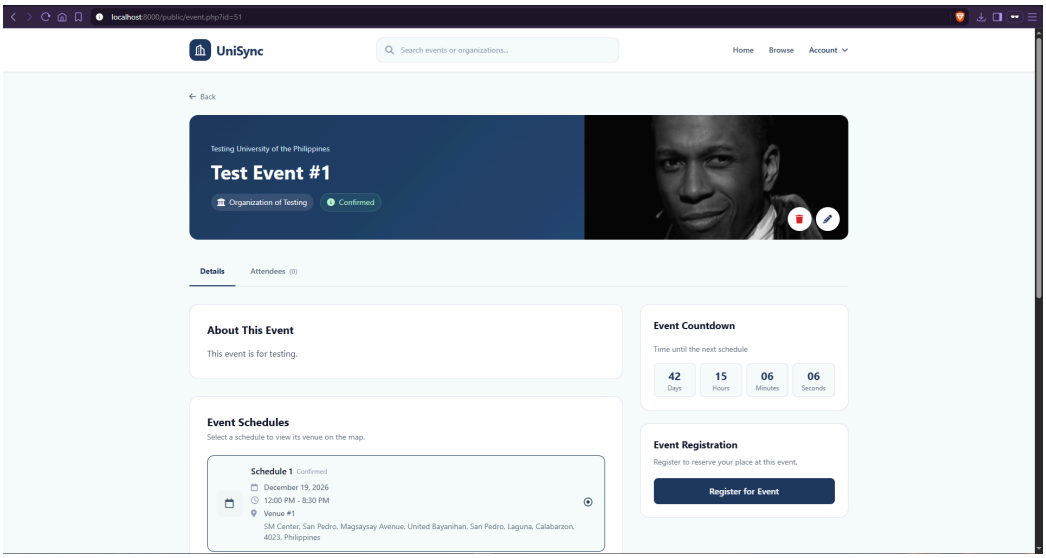


Figure 4. Event with schedule and venue

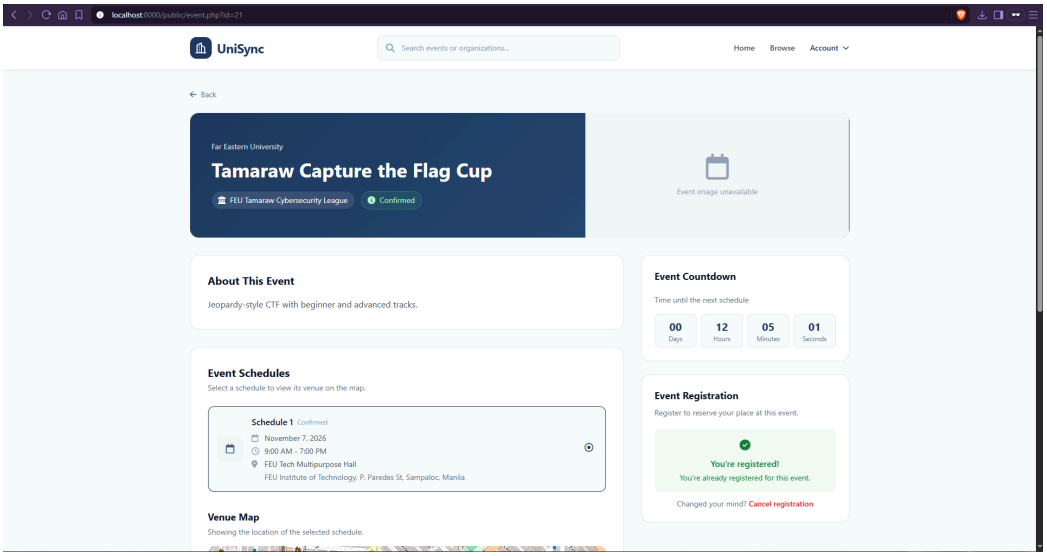


Figure 5. Event registration and cancel option

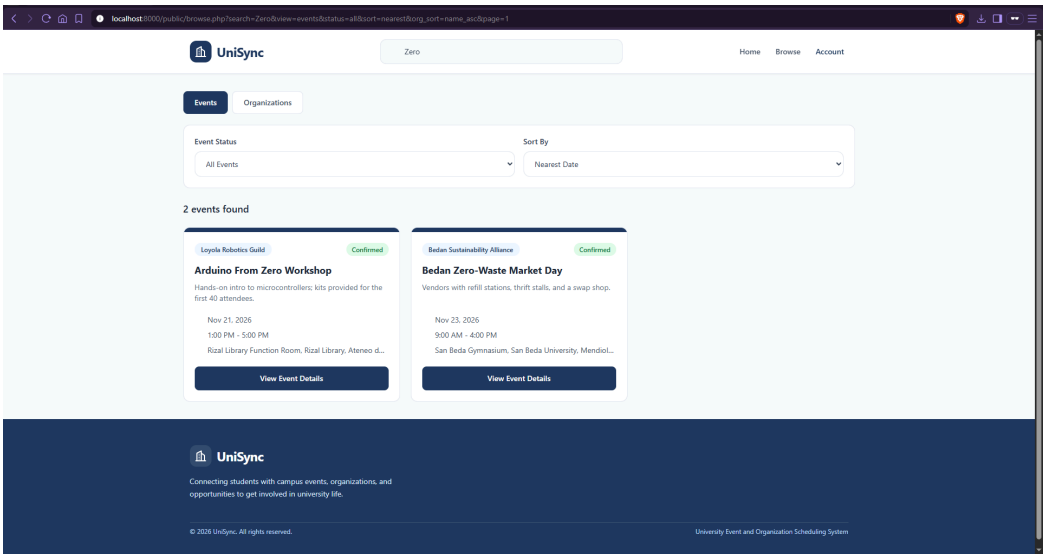


Figure 6. Browse and search results

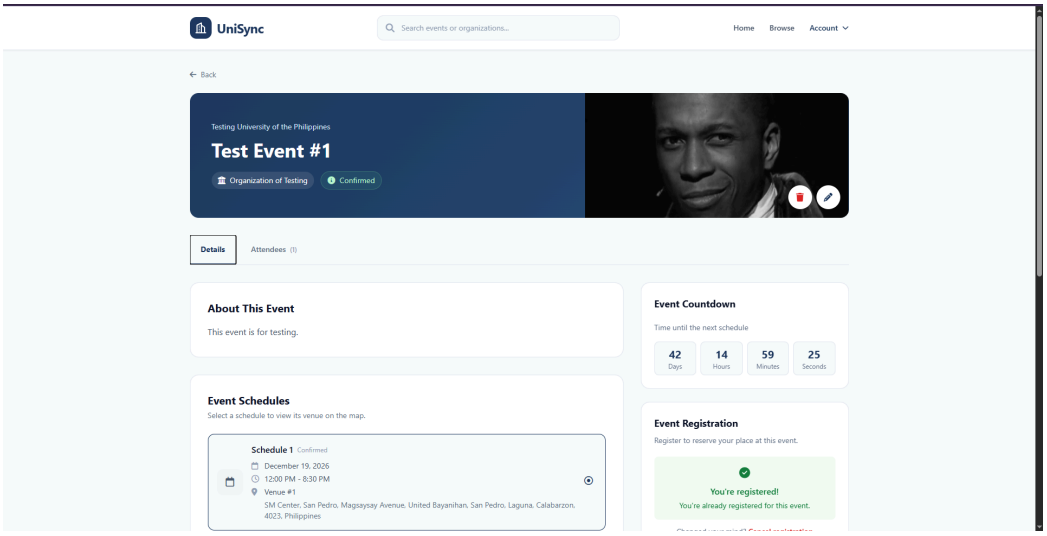


Figure 7. Event page with countdown

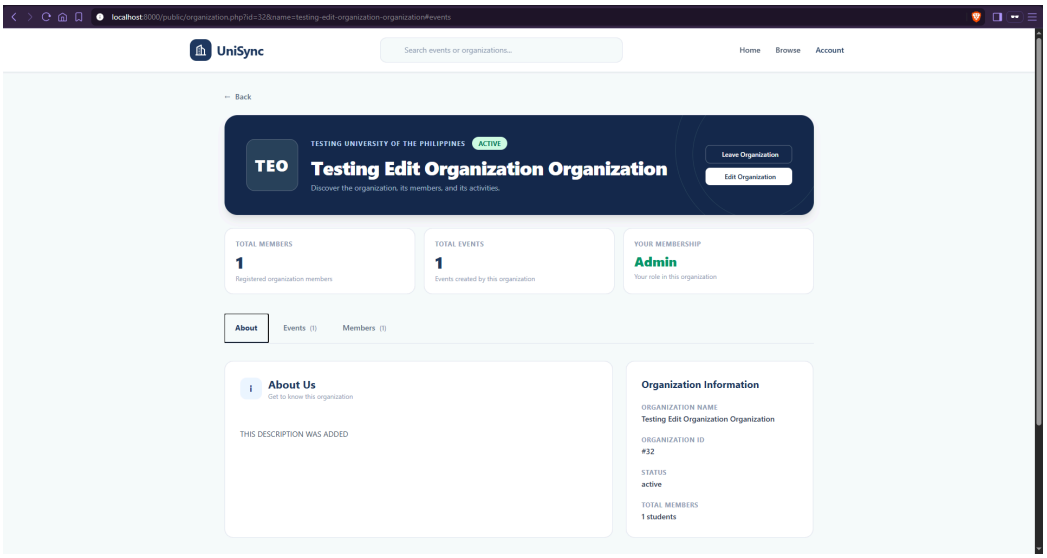


Figure 8. Organization edited; description saved

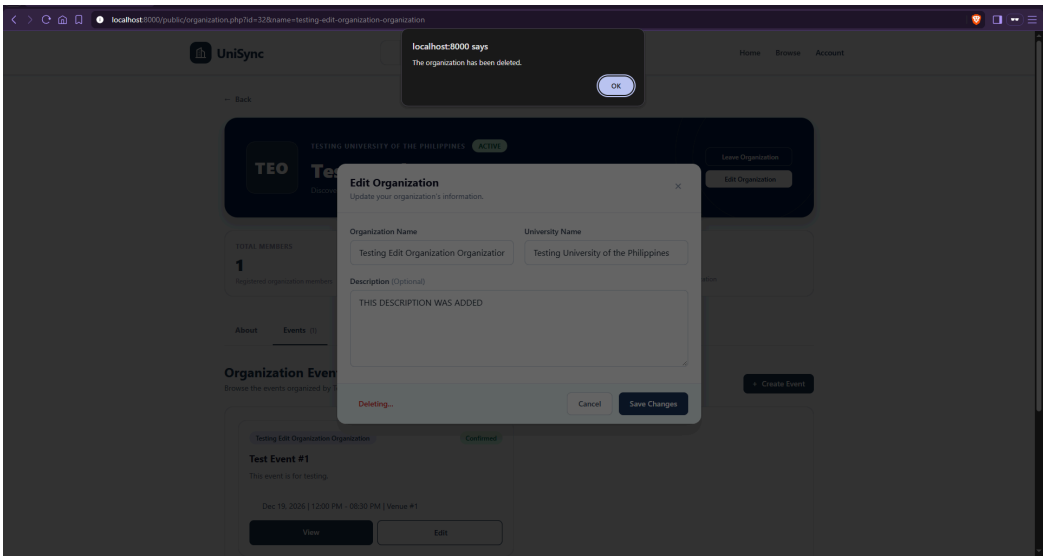


Figure 9. Organization deletion confirmed

6. Setup and Configuration

Full instructions are in the repository README. **Prerequisites:** PHP 8.1+ with `pdo_pgsql`, `fileinfo` and `mbstring`; PostgreSQL 18; Git; a browser with internet access (map tiles and icons load online).

```
git clone <repository-url>; cd unisync
psql -U postgres -c "CREATE DATABASE grp10_unisyncdb;"
psql -U postgres -d grp10_unisyncdb -f db/init.sql
psql -U postgres -d grp10_unisyncdb -f db/seed.sql
cp .env.example .env        # fill in local values
php -S localhost:8000       # then open http://localhost:8000/
```

Enable the PHP extensions in `php.ini` and make `public/uploads/events/` writable.

After pulling any update, re-run db/init.sql and then db/seed.sql before testing. This clears data created during testing and restores the sample data. If the unisync_app runtime user has not been created, the local

postgres account can be used for testing. Common errors and fixes are listed in the README and docs/troubleshooting.md.

Variable	Description	Placeholder value
DB_HOST	PostgreSQL host	localhost
DB_PORT	PostgreSQL port	5432
DB_NAME	Database name	grp10_unisyncdb
DB_USER	Runtime database user	unisync_app
DB_PASSWORD	Password for that user	your_password_here

Database user and group role setup is documented in `docs/database-roles.md`. Demo accounts for the seeded data are listed in the README. No real passwords or secrets are committed; `.env` is excluded from the repository.

7. Issues and Resolutions

Issue	Resolution
Clearing the organization filter on the browse page showed events from other organizations	Fixed the filter logic so clearing the filter resets the selection without showing events outside the intended organization
Older database used a STUDENT table instead of ACCOUNT	Schema reworked so fresh installs create ACCOUNT directly in `db/init.sql`; the temporary migration script was removed
Cluttered project structure (unused scripts, TODO file, empty folders, homepage in public/index.php)	Cleanup done; root `index.php` is the entry point, homepage moved to `public/home.php`, static files under `public/assets/`
Application required users to open public/index.php instead of the project root	Added a root-level `index.php` entry point so the app opens from the project root
Outdated database after pulling an update caused missing-column and rejected-status errors	Re-run db/init.sql, then db/seed.sql after every update (this clears test data and restores the sample data); documented in the README under Updating to a New Version

8. Cross-Group Verification

Projects tested by this group, one row per project and activity, mirroring the class tracker.

Project (activity)	Work performed	Issues / findings
01 DropBoxed (A3)	Followed the README, set up the database, loaded sample data, launched the app, ran db:verify and test:smoke, and tested Resident, Staff and Admin roles.	Missing `.env` does not stop `npm run db:setup`; it reports success, uses the default database and fails to configure the `dropboxed_app` password.
01 DropBoxed (A4)	Inspected roles, memberships, and database and table privileges; tested runtime and read-only restrictions with SQL scripts.	Access controls worked as expected. Minor cross-database connectivity from PostgreSQL's default PUBLIC privileges.
01 DropBoxed (A5)	Checked health, reviewed logs, ran ANALYZE and VACUUM, evaluated a query with EXPLAIN.	Healthy. Duplicate venue entries and an invalid test query in the logs. Sequential scan reasonable for the small table.

Project (activity)	Work performed	Issues / findings
01 DropBoxed (A6)	Custom-format backup, archive verified, restored to a separate database, verified tables, records, constraints, roles; controlled recovery simulation.	Backup and restore successful. 8 tables, 72 constraints and role privileges verified. Deleted test data recovered; original database unchanged.
02 Eventer / HSEMS (A3)	Followed the README, configured PostgreSQL and roles, launched backend and frontend, tested Super Admin and Event Organizer roles and persistence.	Initial startup failed: bootstrap passwords did not meet the 10-character minimum. Correcting local configuration resolved it.
02 Eventer / HSEMS (A4)	Verified roles, schema ownership, table privileges, audit log restrictions and application access.	Success. `event_app` cannot modify audit logs or schema objects and has read-only access to the role table.
02 Eventer / HSEMS (A5)	Checked size, table statistics, version, connections; ANALYZE and VACUUM; EXPLAIN ANALYZE.	Success. Maintenance completed; tested query ran in 0.102 ms with no significant issues.
02 Eventer / HSEMS (A6)	Custom-format backup, archive inspected, restored to a separate database, verified tables and row counts.	Success, no reported errors. All 10 tables present.
03 PAWPAW (A3)	Application setup, authentication and functional testing of core features.	Success. No major issues.
03 PAWPAW (A4)	Created roles, assigned restricted runtime privileges, tested access.	Runtime login worked; schema access and SELECT allowed; TRUNCATE and CREATE TABLE denied.
03 PAWPAW (A5)	Inspected health, connections, Activity 4 roles, logging; EXPLAIN on an application query.	Database accessible, roles verified. Log directory inspection and ANALYZE denied; VACUUM not performed. Sequential scan on appointment.
03 PAWPAW (A6)	Created and inspected a backup, restored to a separate database, verified objects and privileges, recovery simulation.	Success. 14 tables, 118 constraints, privileges verified, deleted test record recovered. Application-level verification not performed.
04 HBCMS (A3)	Followed the README, ran SQL scripts, launched the app; tested Patient and Admin login, appointment creation and doctor management.	`update_note_unique.sql` hit an existing relation; Staff login rejected; Admin doctor changes did not persist after refresh.
04 HBCMS (A4)	Inspected roles, memberships and privileges; tested an UPDATE using `hb_app`.	`hb_app` inherits `hb_staff`; UPDATE on doctor records correctly denied; no unexpected privileges.
04 HBCMS (A5)	Inspected PostgreSQL 18.6, size, 11 tables, connections, logs; EXPLAIN on the doctor-list query; checked maintenance privileges.	Logs showed authentication failures and permission errors from other accounts and schemas. `hb_app` lacks MAINTAIN on all 11 tables, so ANALYZE and VACUUM could not run.
04 HBCMS (A6)	Custom-format backup, archive inspected, restored to a separate database; verified tables, row counts, constraints.	Success. 11 tables, row counts matched, 90 constraints. Application-level and per-role verification not performed.
05 SHIORI Café (A3)	Set up the system and tested its general functionality, including reservation, ordering, and Staff Counter POS operations.	Most tested functions worked as intended. Receipt completion failed due to a NOT NULL constraint violation involving reservation_id.
05 SHIORI Café (A4)	Inspected database roles, role memberships, database and schema privileges, and tested application runtime access.	Runtime role lacked schema USAGE and table privileges. All 18 tables denied SELECT, INSERT, UPDATE, and DELETE. Actual access test returned a permission-denied error.
05 SHIORI Café (A5)	Inspected database health, sessions, objects, roles, and PostgreSQL logs. Executed ANALYZE and VACUUM on the reservation table and attempted query plan analysis.	Maintenance completed using postgres. Query analysis under app_runtime was blocked because it lacks schema USAGE privileges. Logs also contained the previously identified Counter POS receipt error.

Project (activity)	Work performed	Issues / findings
05 SHIORI Café (A6)	Created and inspected a custom-format backup, restored it to a separate database, verified database objects and records, checked access control, and performed a controlled recovery simulation.	Backup, restoration, and recovery simulation succeeded. Existing runtime schema access restriction remained.
06 ByaheRentals (A3)	Created the database, ran schema and seed scripts, applied six migrations, configured roles and grants, launched the Spring Boot app; tested customer features.	`schema.sql` failed to create `inspection_item` because no schema was selected; created manually under `public`. Everything else passed.
06 ByaheRentals (A4)	Inspected roles and memberships; checked table privileges against the access-control SQL.	Assignments confirmed. Read-only role has SELECT on 15 tables. Runtime DELETE on `vehicle` confirmed as intentional.
06 ByaheRentals (A5)	Inspected health, connections, statistics, role memberships, logging, and a rental query plan; attempted ANALYZE and VACUUM on `public.customer`.	Both maintenance operations skipped for insufficient permissions; log files not readable by the application account. The rental query used `idx_rental_customer`.
06 ByaheRentals (A6)	Custom-format backup, restored to a separate database, verified tables, records, constraints, indexes, roles and permissions; recovery simulation with a second backup.	Deleted test record recovered; original unchanged. Role definitions are not part of the database backup; roles were verified on the same instance.
07 Applicant AMS (A3)	Configured environment variables, imported schema scripts, started the PHP built-in server; tested applicant registration and admin status updating.	500 Internal Server Error when creating scheduled interviews as Administrator or Staff.
07 Applicant AMS (A4)	Inspected existing PostgreSQL roles, memberships, database ownership, schema privileges, table ACLs, and effective applicant table privileges.	All three group roles exist, but none has effective privileges on the applicant table. No access-control changes were made.
07 Applicant AMS (A5)	Inspected database health, sessions, roles, indexes, and PostgreSQL logs. Performed VACUUM with ANALYZE and examined an applicant lookup query execution plan.	PostgreSQL logs contained authentication failures, a permission error, and autovacuum warnings, but these could not be attributed to Group 07. The applicant lookup used a sequential scan on a small table and completed in 0.042 ms.
07 Applicant AMS (A6)	Created and inspected a PostgreSQL custom-format backup, restored it into a separate test database, verified 9 tables, sample records, and 70 constraints, tested backup-account permissions, and completed a recovery simulation using a separate recovery snapshot.	Existing Activity 4 group roles had no effective SELECT or INSERT privileges on the applicant table. The dedicated backup account successfully performed SELECT but was denied UPDATE. The disposable recovery record was successfully restored.
08 Catering Business Management System (A3)	Installed dependencies, configured environment variables, created the PostgreSQL database, imported schema and seed data, started the backend, and tested admin login, staff dashboard, customer registration/login, and reservation submission.	No issues encountered during the tested setup and basic functionality checks.
08 Catering Business Management System (A4)	Created three group roles and three login roles, assigned memberships, and granted database, schema, table, and sequence privileges. Verified Administrator, Runtime, and Viewer permissions, including a successful Viewer SELECT and denied INSERT.	No issues encountered during role and privilege verification.

Project (activity)	Work performed	Issues / findings
08 Catering Business Management System (A5)	Inspected database size, table statistics, dead tuples, and existing indexes. Executed VACUUM (ANALYZE) and verified all 12 tables had zero dead tuples. Analyzed reservation query performance and checked database connections.	No issues encountered during database maintenance and query analysis.
08 Catering Business Management System (A6)	Created and inspected a PostgreSQL custom-format backup, recorded its SHA256 checksum, and restored it into a separate recovery database. Verified all 12 tables, matching record counts, and 81 constraints.	No issues encountered during backup creation and recovery verification.
09 Batumbakal Gym Membership Management System (A3)	Set up the database, imported schema and seed data, applied migration, launched the application, and tested all eight modules, including Create, Edit, and Delete operations.	Create operations involving related records are difficult to use because valid foreign key IDs must be supplied manually. Invalid references result in HTTP 500 errors instead of user-friendly validation. Edit and Delete operations worked as expected.
09 Batumbakal Gym Membership Management System (A4)	Created database access-control roles, assigned login memberships, configured table and sequence privileges, and tested admin, runtime, and read-only permissions.	All tested permissions passed. No blocking issues encountered.
09 Batumbakal Gym Membership Management System (A5)	Checked database size and table statistics, executed VACUUM ANALYZE, verified dead tuple cleanup, inspected indexes, and analyzed a representative membership query using EXPLAIN ANALYZE.	All 8 tables showed zero dead tuples after maintenance. The analyzed query returned 4 rows with an execution time of 0.109 ms. No blocking issues encountered.
09 Batumbakal Gym Membership Management System (A6)	Created a custom-format PostgreSQL backup, inspected its archive contents, recorded its SHA256 checksum, restored it to a temporary database, and compared original and restored table counts.	All 8 tables and 35 records matched. Backup and restoration completed successfully.
11 ReserveIT: Classroom and Material Reservation System (A3)	Cloned the repository, installed dependencies, configured the environment, created database roles, initialized the schema, seeded sample data, verified the application database connection, and checked application access using the documented accounts.	Initial database creation used lowercase reserveit instead of the required case-sensitive ReserveIT. Recreated the database with the correct capitalization. Subsequent setup and application checks worked as intended.
11 ReserveIT: Classroom and Material Reservation System (A4)	Reviewed existing role and grant scripts; verified login/group-role attributes and memberships; tested runtime, reporter, and DBA permissions; tested database creation restriction and schema creation restriction; verified cross-database connection behavior.	Runtime, reporter, and DBA permissions behaved as intended in tested operations. However, reserveit_app could connect to UniSync's grp10_unisyncdb because PUBLIC has CONNECT permission. Actual connection was confirmed. No privileges were changed.
11 ReserveIT: Classroom and Material Reservation System (A5)	Inspected database health, table statistics, transaction activity, performed ANALYZE and VACUUM, tested query performance using EXPLAIN ANALYZE, and reviewed PostgreSQL logs.	reserveit_dba lacked privileges for application-table maintenance; maintenance succeeded using reserveit_owner. Logs contained authentication failures, SQL errors, and permission warnings, including unrelated server-wide activity.
11 ReserveIT: Classroom and Material Reservation System (A6)	Created a custom-format backup, restored it into a separate recovery database, and verified 8 tables, 37 records, constraints, indexes, privileges, and application access.	Existing column-level UPDATE grants on five room columns were found in both the original and restored databases.

Feedback received on UniSync and fixes made

Tested by	Issue reported	Fix	Verification result
Group 05 (A4)	Re-running `db/init.sql` drops and recreates the tables, which also removes previously granted privileges.	Documented that the role and grant scripts must be re-applied after the database is re-initialized.	Sequence synchronization verified to prevent duplicate-key errors (per Group 05's Activity 4 report).
Group 05 (A5)	Database found healthy; ANALYZE, VACUUM and REINDEX completed; event-attendee query used index scans.	No change required.	No immediate performance changes recommended (per Group 05's feedback of 7 November 2026).

9. AI Usage Declaration

AI tool	Tasks performed	How the output was validated
Claude	Coding assistance for the PHP backend and frontend, jQuery/AJAX, database connectivity, CRUD, authentication, organization and event management, debugging and error fixing.	Reviewed the suggested code, integrated it, tested the affected features and corrected errors found during implementation.
Claude	ERD preparation, database design, and creation of database structures and scripts.	Reviewed the ERD against the approved blueprint; checked tables, relationships, keys and constraints.
Gemini	Explanations of programming concepts, database concepts, SQL operations and implementation approaches.	Compared explanations with the project's requirements and implementation; rejected suggestions that did not match the approved design.
ChatGPT	PHP and PostgreSQL troubleshooting, jQuery/AJAX debugging, SQL and schema review, workflow planning, README and technical and testing documentation.	Tested proposed solutions in the running application, reviewed SQL scripts, checked database behavior, verified CRUD and user roles, and corrected suggestions that did not match the project.

The group acknowledges that AI tools played a significant role in developing UniSync, particularly in generating code, suggesting implementations, debugging and explaining technical concepts. Members provided requirements, reviewed generated solutions, integrated code and tested the result. AI output was not accepted blindly: for example, the organization filter issue was found during testing and the filtering logic was corrected, and database configuration and access-control suggestions were reviewed against the intended security requirements. Members acknowledge that their understanding of some implementation details is still developing.